VACF Module Specification

API and Algorithm Standard for Velocity Autocorrelation Analysis

mdstats

2026-07-22

1. Purpose

This document specifies `vacf.py`, the velocity autocorrelation function (VACF) module for `mdstats`.

The module consumes a time-ordered `AtomisticFrameCollection` and computes the **self velocity autocorrelation** of selected atoms. Its output is the common foundation for later modules that calculate:

- normalized velocity-memory curves;
- self-diffusion through Green-Kubo integration;
- VACF-derived consistency checks against MSD;
- velocity power spectra;
- vibrational and phonon density-of-states estimates;
- species-, atom-, and direction-projected dynamics.

The first implementation returns raw correlation data only. Windowing, smoothing, spectral transformation, tail truncation, transport integration, and frequency fitting are separate, explicit operations.

2. Scope and non-goals

The module computes a **self** correlation:

$$C_{\mathrm{self}}(t) = \sum_i \left\langle \mathbf{v}_i(t_0) \cdot \mathbf{v}_i(t_0 + t) \right\rangle_{t_0}.$$

It does not include distinct-particle terms with $i \neq j$. Therefore, the first version does not directly calculate:

- charge-current autocorrelation or ionic conductivity;
- Onsager cross coefficients;
- collective diffusion;
- heat-current autocorrelation or thermal conductivity;
- wavevector-resolved phonon dispersion;
- mode-projected phonon lifetimes.

Those quantities require collective or mode-resolved correlations and belong in later modules.

3. Dependency on `AtomisticFrameCollection`

VACF is a temporal observable. The input must satisfy

```
collection.frame_semantics is FrameSemantics.TRAJECTORY
```

An independent ensemble is invalid even if it contains source time labels or stored velocity arrays. Frame order and time separation must represent one continuous physical trajectory.

The implementation must call:

```
collection.require_trajectory("VACF")
collection.require_minimum_frames(2, "VACF")
times = collection.require_time_axis("VACF")
velocities = collection.require_velocities("VACF")
```

The module assumes the parser and preprocessor have already enforced:

- a fixed atom count and persistent atom ordering;

- constant atomic species and masses;
- Cartesian velocities in A/ps;
- frame times in ps;
- finite `float64` arrays;
- explicit trajectory semantics.

The relevant collection fields are conceptually:

```
class AtomisticFrameCollection:
    frame_semantics: FrameSemantics
    frame_ids: NDArray[np.int64]              # (T,)

    atomic_numbers: NDArray[np.int32]         # (N,)
    masses: NDArray[np.float64]               # (N,), amu

    times: NDArray[np.float64]                # (T,), ps
    velocities: NDArray[np.float64]           # (T, N, 3), A/ps

    provenance: FrameCollectionProvenance | None
    metadata: dict[str, Any]
```

Cells and positions are not required for the base VACF once Cartesian velocities are available.

3.1 Shared input preparation

As of `mdstats 0.19.6a0`, the validated time-grid, velocity, selection, weighting, drift, and per-atom mapping behavior is implemented once in `mdstats.analysis._velocity_common`. `compute_vacf()` consumes the resolved private `VelocityInputBundle`; the public VACF API and numerical estimator are unchanged. See `_velocity_common_spec.md` for the normative shared contract.

4. Time-grid convention

The first implementation requires a uniformly sampled, strictly increasing time axis:

$$t_n = t_0 + n\Delta t.$$

For a frame lag k,

$$\tau_k = k\Delta t.$$

`lag_steps` means **frame lag**, not the difference between source integration step labels. `lag_times` is the authoritative physical separation in ps.

Nonuniform sampling must be rejected. Correct treatment would require a special irregular-time estimator or explicit resampling, neither of which is part of the first implementation.

5. Foundational VACF definition

Let A be the selected atom set, w_i a nonnegative atom weight, and $\mathcal{O}_k$ the set of valid time origins for frame lag k.

The canonical tensor stored by the module is the raw weighted self-correlation sum, averaged over time origins:

$$S_{\alpha\beta}[k] = \frac{1}{N_{\mathrm{orig}}(k)} \sum_{n \in \mathcal{O}_k} \sum_{i \in A} w_i \, v_{i\alpha}[n] \, v_{i\beta}[n+k].$$

Here

$$N_{\mathrm{orig}}(k) = |\mathcal{O}_k|.$$

The Cartesian component correlations are the tensor diagonal:

$$S_x = S_{xx}, \qquad S_y = S_{yy}, \qquad S_z = S_{zz}.$$

The scalar dot-product correlation is the trace:

$$S[k] = \mathrm{Tr}\ S_{\alpha\beta}[k] = S_x[k] + S_y[k] + S_z[k].$$

Only nonnegative lags are stored.

5.1 Tensor orientation

The tensor convention is

$$S_{\alpha\beta}[k] \propto v_\alpha[n] v_\beta[n+k].$$

At positive lag, finite-sample cross components need not be symmetric:

$$S_{\alpha\beta}[k] \neq S_{\beta\alpha}[k].$$

The implementation must retain the full tensor and must not silently symmetrize it.

6. Weighting conventions

Weighting and normalization are separate concepts.

6.1 Uniform weighting

For

$$w_i = 1,$$

the raw sum represents the total selected-atom self correlation. Dividing by

$$W = \sum_i w_i = N_A$$

gives the per-atom VACF used for self-diffusion.

6.2 Mass weighting

For

$$w_i = m_i,$$

the correlation is mass weighted:

$$S_m(t) = \left\langle \sum_i m_i \mathbf{v}_i(0) \cdot \mathbf{v}_i(t) \right\rangle.$$

This form has the cleanest classical harmonic relation to the vibrational density of states.

6.3 Explicit weights

The user may provide one weight for each selected atom. Explicit weights must be:

- one-dimensional with shape `(n_selected_atoms,)`;
- finite;
- nonnegative;
- not all zero.

Signed weights are excluded from the foundational self-VACF. They belong in future current-correlation and projection modules.

6.4 Stored and derived forms

The canonical result stores the raw weighted sum S and the weight sum

$$W = \sum_i w_i.$$

A weighted mean is derived as

$$\overline{C}(t) = \frac{S(t)}{W}.$$

For uniform weights, this is the ordinary per-atom VACF. Raw and mean forms remain recoverable from each other.

7. Normalization convention

The normalized VACF is

$$Z(t) = \frac{C(t)}{C(0)}, \qquad Z(0) = 1.$$

Normalization is useful for comparing relaxation shapes, but it discards the amplitude needed for:

- diffusion coefficients;
- equipartition checks;
- physically normalized spectral intensities;
- comparison of thermal velocity scales.

Therefore, normalized VACF is a derived view and is never the canonical stored observable.

A normalization method must raise a descriptive error when the lag-zero amplitude is zero or numerically indistinguishable from zero.

8. Drift removal

A uniform framewise velocity drift creates a long-lived low-frequency contribution. For a reference atom set R, the center-of-geometry velocity is

$$\mathbf{V}_R^{\mathrm{geom}}(t) = \frac{1}{|R|} \sum_{j \in R} \mathbf{v}_j(t),$$

and the center-of-mass velocity is

$$\mathbf{V}_R^{\mathrm{COM}}(t) = \frac{\sum_{j \in R} m_j \mathbf{v}_j(t)}{\sum_{j \in R} m_j}.$$

Corrected velocities are

$$\mathbf{v}'_i(t) = \mathbf{v}_i(t) - \mathbf{V}_R(t).$$

Drift removal is optional. The drift reference selection is independent of the measured atom selection.

The module must not automatically subtract the temporal mean velocity of each atom. Such a transformation can remove real persistent transport.

9. Public API

The module file is

`mdstats/analysis/vacf.py`

The public entry point is exported as `mdstats.compute_vacf`.

```
from __future__ import annotations

from typing import Literal

from numpy.typing import ArrayLike


def compute_vacf(
```

```
    collection: AtomisticFrameCollection,
    *,
    species: SpeciesSelection = None,
    atom_indices: ArrayLike | None = None,
    max_lag: int | None = None,
    origin_stride: int = 1,
    lag_stride: int = 1,
    weights: Literal["uniform", "mass"] | ArrayLike = "uniform",
    drift_mode: Literal[
        "center_of_mass",
        "center_of_geometry",
    ] | None = None,
    drift_species: SpeciesSelection = None,
    drift_atom_indices: ArrayLike | None = None,
    compute_tensor: bool = True,
    per_atom: bool = False,
    per_atom_indices: ArrayLike | None = None,
    backend: Literal["auto", "direct", "fft"] = "auto",
    atom_block_size: int | None = None,
) -> VACFResult:
    ...
```

9.1 Atom selection

`species` and `atom_indices` are mutually exclusive and use the shared `resolve_atom_selection()` utility.

- neither supplied: select all atoms;
- `species="Na"`: select all Na atoms;
- `species=["Na", "K"]`: select both species;
- `atom_indices=[2, 7, 18]`: select canonical atom indices.

The selection must be nonempty and contain no duplicate or out-of-range indices.

9.2 Lag controls

`max_lag` is the largest included frame lag. The default is

$$k_{\max} = \left\lfloor \frac{T}{2} \right\rfloor.$$

Lag zero is always included. Returned lags are

```
np.arange(0, max_lag + 1, lag_stride)
```

with `lag_stride >= 1`.

`origin_stride` controls the spacing between time origins in the direct estimator. It must be at least one.

9.3 Per-atom output

`per_atom=True` requests correlations for every measured atom.

`per_atom_indices` requests output only for specified canonical atom indices. The indices must be a subset of the measured atom selection. Supplying `per_atom_indices` implicitly enables per-atom output.

Per-atom output is opt-in because its storage scales as

$$O(N_{\text{output}} L).$$

10. Result data structure

```
from dataclasses import dataclass, field
from typing import Any

import numpy as np
```

```python
from numpy.typing import NDArray


FloatArray = NDArray[np.float64]
IntArray = NDArray[np.int64]


@dataclass(frozen=True, slots=True)
class VACFResult:
    lag_steps: IntArray                    # (L,), frame lags
    lag_times: FloatArray                  # (L,), ps

    scalar_sum: FloatArray                 # (L,)
    components_sum: FloatArray             # (L, 3)
    tensor_sum: FloatArray | None           # (L, 3, 3)

    per_atom_scalar: FloatArray | None      # (L, M)
    per_atom_components: FloatArray | None  # (L, M, 3)
    per_atom_indices: IntArray | None       # (M,)

    n_origins: IntArray                    # (L,)

    atom_indices: IntArray                 # (N_A,)
    atom_weights: FloatArray               # (N_A,)
    weight_sum: float

    weighting: str
    drift_mode: str | None
    backend: str

    metadata: Mapping[str, Any] = field(default_factory=dict)
    signature: DynamicsInputSignature | None = None
```

10.1 Shape and identity constraints

The result must satisfy

```
scalar_sum.shape            == (L,)
components_sum.shape        == (L, 3)
tensor_sum.shape            == (L, 3, 3) or None
n_origins.shape             == (L,)
atom_indices.shape          == (N_A,)
atom_weights.shape          == (N_A,)
```

and

$$S[k] = S_x[k] + S_y[k] + S_z[k].$$

When `tensor_sum` is stored,

$$(S_x, S_y, S_z) = \operatorname{diag} S_{\alpha\beta}.$$

The tensor is not required to be symmetric for $k > 0$.

Per-atom fields store the weighted contribution of each requested output atom:

$$S_i[k] = \frac{w_i}{N_{\text{orig}}(k)} \sum_{n \in \mathcal{O}_k} \mathbf{v}_i[n] \cdot \mathbf{v}_i[n+k].$$

If all measured atoms are returned, summing `per_atom_scalar` over atoms must reproduce `scalar_sum`.

10.2 Derived result views

The result should expose at least:

```
result.scalar_mean
result.components_mean
result.tensor_mean
result.normalized_scalar()
result.normalized_components()
result.project_direction(direction)
```

where mean fields divide by `weight_sum`.

For a unit direction $\hat{\mathbf{n}}$,

$$S_{\hat{\mathbf{n}}}(t) = \hat{\mathbf{n}}^T S(t) \hat{\mathbf{n}}.$$

Directional projection requires `tensor_sum`.

11. Direct estimator

For each requested lag k, direct evaluation uses the time-origin set

$$\mathcal{O}_k = \{0, s_o, 2s_o, \ldots < T - k\},$$

where s_o is `origin_stride`.

The number of origins is

$$N_{\mathrm{orig}}(k) = \left\lfloor \frac{T - k - 1}{s_o} \right\rfloor + 1.$$

Pseudocode:

```
velocities <- selected Cartesian velocities
velocities <- optional framewise drift correction
weights    <- validated selected-atom weights
lags       <- 0, lag_stride, ..., max_lag

for each lag k:
    origins <- 0, origin_stride, ... < T-k
    v0      <- velocities[origins]
    vk      <- velocities[origins + k]

    components_sum[k] <- mean over origins of
                         sum over atoms of
                         weights * v0 * vk

    if compute_tensor:
        tensor_sum[k] <- mean over origins of
                         sum over atoms of
                         weights * outer(v0, vk)

    scalar_sum[k] <- trace or sum of components
```

The direct method is the reference implementation because it is transparent and supports arbitrary origin stride.

Its approximate cost is

$$O(N_A T L)$$

and becomes expensive for long MLFF trajectories.

12. FFT estimator

For FFT evaluation, define weighted velocities

$$u_{i\alpha}[n] = \sqrt{w_i}\, v_{i\alpha}[n].$$

For each atom and component pair,

$$R_{i,\alpha\beta}[k] = \mathcal{F}^{-1}\left[\mathcal{F}(u_{i\alpha})^{*}\mathcal{F}(u_{i\beta})\right][k].$$

The implementation must zero-pad to at least

$$N_{\mathrm{FFT}} \geq 2T - 1$$

to avoid circular correlation. A fast transform length may be chosen with `scipy.fft.next_fast_len()`.

After summing atom contributions, divide by the exact pair count:

$$S_{\alpha\beta}[k] = \frac{\sum_i R_{i,\alpha\beta}[k]}{T - k}.$$

The FFT orientation must reproduce the direct convention $v_{\alpha}[n]v_{\beta}[n+k]$ and must be verified by unit tests.

12.1 No cross-atom terms

The implementation must autocorrelate each atom before summing. It must not autocorrelate the atom-averaged velocity, because

$$\left(\sum_i \mathbf{v}_i(0)\right) \cdot \left(\sum_j \mathbf{v}_j(t)\right)$$

contains unwanted $i \neq j$ collective terms.

12.2 Atom blocking

FFT transforms should be processed in atom blocks:

```
initialize accumulated spectra/correlations
for each atom block:
    load block velocities
    apply sqrt(weights)
    FFT along time
    accumulate component or tensor products
    discard block transforms
inverse transform accumulated products
normalize by T-k
```

This limits working memory to approximately

$$O(BN_{\mathrm{FFT}})$$

for block size B rather than storing transforms for every atom.

12.3 FFT restrictions

The standard FFT estimator uses every time origin. Therefore:

- `origin_stride == 1` is required for `backend="fft"`;
- `lag_stride` is applied after computing the complete correlation;
- nonuniform time grids are rejected.

13. Backend selection

Supported values are:

```
backend="direct"
backend="fft"
backend="auto"
```

`direct` and `fft` must implement the same estimator whenever `origin_stride == 1`.

`auto` chooses a backend from estimated computational work:

- use direct when `origin_stride != 1`;
- prefer direct for small problems;
- prefer FFT when $N_A T L$ is large;
- use the shared FFT planner to select an atom block size that respects a conservative memory target once the FFT backend has been chosen.

The exact heuristic is an internal implementation detail, not part of the numerical definition. The chosen backend and block size must be recorded in `VACFResult.metadata`.

14. Numerical units

Internal velocities have units

$$[\mathbf{v}] = \mathrm{A/ps} \ .$$

For uniform or dimensionless explicit weights,

$$[S] = \mathrm{A}^2/\mathrm{ps}^2.$$

For mass weighting,

$$[S_m] = \mathrm{amu}\,\mathrm{A}^2/\mathrm{ps}^2.$$

Integrating the uniform per-atom VACF over time produces

$$\mathrm{A}^2/\mathrm{ps},$$

which is a diffusion unit.

The result metadata must record the velocity and correlation units.

15. Relation to diffusion and MSD

For the uniform per-atom scalar VACF $C(t)$ in d dimensions,

$$D = \frac{1}{d}\int_0^\infty C(t)\,dt.$$

The time-averaged MSD satisfies

$$\mathrm{MSD}(t) = 2\int_0^t (t-\tau)C(\tau)\,d\tau.$$

The VACF module does not perform these integrations. Separate transport functions provide:

```
integrate_vacf_to_diffusion(result)
reconstruct_msd_from_vacf(result)
```

Direct position-based MSD remains an independent primary observable because:

- fixed-origin MSD is not recoverable from a stationary VACF;
- long-time VACF noise accumulates during integration;
- reconstructed velocities can attenuate high-frequency motion;
- independent MSD and VACF implementations provide a valuable consistency check.

16. Relation to velocity spectra and VDOS

For a real equilibrium VACF,

$$S(\omega) = 2\int_0^\infty C(t)\cos(\omega t)\,dt.$$

However, finite-sample spectral estimation introduces independent choices:

- window function;
- zero padding;
- periodogram versus VACF transform;
- Welch segmentation;
- frequency units;
- mass and degree-of-freedom normalization.

Therefore, `compute_vacf()` does not calculate a spectrum. Future modules will provide:

```
compute_velocity_spectrum(...)
compute_vdos(...)
```

They may share low-level FFT machinery without being forced to transform the final displayed VACF estimator.

17. Validation rules

The implementation must reject:

- ensemble semantics;
- fewer than two frames;
- missing velocities;
- missing physical times;
- nonfinite or non-increasing times;
- nonuniform time spacing;
- nonfinite velocities;
- empty atom selections;
- duplicate or out-of-range atom indices;
- invalid `max_lag`;
- nonpositive strides;
- malformed or all-zero weights;
- explicit weights with the wrong shape;
- `backend="fft"` with `origin_stride != 1`;
- per-atom output indices outside the measured selection;
- nonpositive FFT block sizes.

The implementation should issue warnings for:

- velocities reconstructed by finite difference;
- large requested lags with few contributing origins;
- drift removal using the same small mobile group being measured;
- a lag-zero amplitude too small for reliable normalization;
- unusually sparse saved velocity frames for spectral use.

18. Velocity provenance

When provenance is available, the result must record

`collection.provenance.velocity_source`

Typical values are:

```
native
finite_difference
```

Finite-difference velocities are acceptable for the VACF calculation, but the module should warn that high-frequency amplitudes may be attenuated. Native velocities are strongly preferred for vibrational spectra and linewidths.

19. Stationarity and trajectory slicing

The multiple-time-origin VACF assumes approximate stationarity. A single VACF should not average indiscriminately across:

- melting or crystallization;

- thermal ramps;
- strong structural relaxation;
- changing external fields;
- other nonequilibrium transitions.

The user should first select a physically stationary trajectory interval and then compute VACF on that subset.

An ensemble created from shuffled or clustered frames remains invalid for VACF, regardless of whether the frames originated from one MD run.

20. High-level implementation flow

```
AtomisticFrameCollection
          |
          v
require trajectory + time axis + velocities
          |
          v
validate uniform sampling and lag controls
          |
          v
resolve measured atoms and optional drift reference
          |
          v
copy selected velocities and remove framewise drift
          |
          v
resolve uniform / mass / explicit atom weights
          |
          v
select backend
   +------+------+
   |             |
 direct       blocked FFT
   |             |
   +------+------+
          |
          v
assemble raw weighted scalar/components/tensor
          |
          v
validate identities, units, and metadata
          |
          v
      VACFResult
```

21. Testing requirements

The direct backend is the numerical reference for the FFT backend.

Required tests include:

21.1 Constant velocity

For constant velocity, the VACF is constant at every lag.

21.2 Harmonic signal

For

$$v(t) = v_0 \cos(\omega t),$$

the VACF has cosine behavior with the expected period.

21.3 White-noise velocity
The correlation is concentrated near zero lag in expectation.

21.4 No cross-atom contamination
Two atoms with distinct signals must produce the sum of their self correlations only.

21.5 Weighting
Uniform, mass, and explicit weights must match hand-calculated results.

21.6 Direct versus FFT
The two backends must agree within floating-point tolerance for:

- odd and even frame counts;
- scalar and Cartesian components;
- full nonsymmetric tensors;
- one and many atoms;
- truncated maximum lag;
- uniform, mass, and explicit weights;
- multiple atom block sizes.

21.7 Drift removal
A uniform framewise translation must disappear after COM or geometric drift subtraction.

21.8 Requirement guards
Tests must verify rejection of ensembles, missing velocities, missing times, nonuniform sampling, invalid weights, and incompatible FFT origin stride.

21.9 VACF-MSD relation
For a suitable stationary synthetic trajectory, numerical integration of the uniform per-atom VACF should approximately reproduce direct time-averaged MSD. This is a consistency test, not an implementation dependency.

22. Computational complexity
Let T be the frame count, N_A the selected atom count, and L the number of returned lags.

Backend	Approximate time	Working memory	Main use
Direct	$O(N_A T L)$	$O(N_A T)$ or less	reference, short AIMD, sparse origins
FFT	$O(N_A T \log T)$	$O(BT)$	long classical and MLFF trajectories

Per-atom output adds

$$O(N_{\mathrm{output}} L)$$

result storage.

23. Edge cases and interpretation warnings

23.1 Frozen or constrained atoms
Atoms with zero velocity contribute zero VACF. Component normalization may be undefined when a direction is constrained.

23.2 Thermostat and barostat effects
Strong coupling can alter relaxation, low-frequency motion, and spectral linewidths. Peak positions are often more robust than damping rates.

23.3 Variable cells

The module uses the Cartesian velocities stored by the preprocessor. It does not silently remove affine cell motion. Barostat-related low-frequency motion must be handled by explicit preprocessing or drift choices.

23.4 Finite trajectory length

The number of origins decreases with lag, so the tail is noisy. Long-lag noise can strongly affect Green-Kubo integrals.

23.5 Finite-size effects

A longer trajectory improves frequency resolution but does not add missing long-wavelength modes. Larger simulation cells are required for denser wavevector sampling.

23.6 Reconstructed velocities

Finite-difference velocities suppress motion near the Nyquist frequency. They are useful for approximate low-frequency dynamics but should not be silently treated as equivalent to native velocities.

24. Reproducibility metadata

`VACFResult` and its `metadata` together preserve the information needed to reproduce the estimator. Core numerical arrays such as `atom_weights` and `n_origins` are first-class result fields rather than duplicated inside metadata. Metadata records at least:

```
input frame ID range and number of frames
physical time spacing
measured atom selection
weighting mode and weight/correlation units
velocity source
drift mode and drift reference selection
maximum lag
origin stride
lag stride
requested and chosen backend
FFT length and atom block size when applicable
source format and source files
```

The collection itself supplies the required trajectory semantics, while `VACFResult.atom_weights` and `VACFResult.n_origins` preserve the exact weights and lag-dependent pair counts.

25. Initial module boundaries

The first implementation of `vacf.py` includes:

- self-VACF only;
- raw weighted sums;
- scalar, Cartesian components, and optional full tensor;
- species and canonical-index selection;
- uniform, mass, and explicit nonnegative weights;
- optional per-atom output;
- framewise COM or geometric drift removal;
- direct and blocked-FFT backends;
- strict trajectory and time-grid validation.

The first implementation excludes:

- spectral windows and smoothing;
- velocity power spectra and VDOS;
- diffusion and MSD integration;
- automatic tail truncation;
- uncertainty estimation;
- cross-particle and charge-current correlations;

- wavevector and phonon-eigenvector projections.

26. Design summary

The module is standardized around the following decisions:

1. Store the raw, origin-averaged, weighted self-correlation sum.
2. Keep normalization derived and reversible.
3. Use uniform weights for self-diffusion and mass weights for vibrational analysis.
4. Make direct and FFT backends implement the same estimator.
5. Keep direct MSD and spectral analysis as independent modules.
6. Reject ensembles because VACF requires physical time ordering.

25. H0 dynamics-contract integration

Every computed `VACFResult` carries a complete source-three-dimensional `DynamicsInputSignature`. It identifies the exact frame sequence, normalized trajectory content, measured atoms, drift mode and exact drift-reference atoms, velocity source, and laboratory coordinate semantics.

All public arrays are copied into owned read-only storage. Metadata is recursively frozen, including nested arrays and sequences. A supplied signature must use the same measured atoms and drift mode and must represent the full source 3D subspace. A frozen dataclass with mutable members is not considered immutable.

`origin_stride`, `lag_stride`, `max_lag`, and `atom_block_size` reject booleans. `compute_tensor` and `per_atom` require actual booleans. These rules are shared with other dynamics modules through `_dynamics_common.py`.

A lower-dimensional transport observable is not defined by dividing `scalar_sum`. Consumers select an explicit axis subset or orthonormal projection. A rotated projection requires `tensor_sum`; otherwise the consumer rejects it. See `_dynamics_common_spec.md` and `vacf_transport_spec.md`.